Kevin Küster

Zusammenfassung: SE_Klausur

Software Engineering

Definition

Programmierung im Großen vs. im Kleinen

Probleme der Programmierung im Großen können nicht durch Lösungen aus dem Bereich des Kleinen gelöst werden

- Programmierung im Kleinen:
  - ↳ Ein Entwickler
  - ↳ Überschaubarer Umfang
  - ↳ Bekannte Algorithmen & Datenstrukturen
  - ↳ Direkte Kontrolle über alles
  - ↳ Kurze Laufzeit
- Programmierung im Großen:
  - ↳ Große Teams
  - ↳ Lange Laufzeit
  - ↳ Millionen Zeilen Code
  - ↳ Kommunikation wird zentral
  - ↳ Anforderungen ändern sich (Laufzeit)
  - ↳ Dokumentation essentiell (Ablauf)
  - ↳ Projektmanagement nötig

Software Engineering

Die Anwendung eines systematischen, disziplinierten, quantifizierbaren Ansatzes auf den Betrieb und die Wartung von Software;
Anwendung Ingenieurswissenschaften auf Software
=> Wirkung: Planbarkeit, Nachvollziehbarkeit, Reproduzierbarkeit

Erfolgsfaktoren + Auswirkungen

- Präzises Anforderungsmanagement:
  - Anforderungen werden vollständig, überprüfbar und eindeutig dokumentiert;
  - Änderungen via geregelter Change-Management-Prozess (Bewertung)
  - Auswirkungen Missachtung: Team baut System, das nicht zum tatsächlichem Bedarf passt
- Geeignetes Vorgehensmodell:
  - Wahl eines zur Projektart passenden Modells;
  - ↳ Sicherheitskritische Systeme: V-Modell
  - ↳ Veränderliche Anforderungen: Scrum
  - Auswirkungen Missachtung: Einbußen in Dokumentationsgenauigkeit oder ineffiziente Arbeitsweise

Gebrauchsnützliche / -taugliche Programme

Gebrauchstauglichkeit ist das Ausmaß, in dem ein Produkt von bestimmten Benutzern in einem bestimmten Nutzungskontext genutzt werden kann zum Erreichen von: Effektivität, Effizienz, Zufriedenheit

Nutzen: Höhere Nutzerakzeptanz. Mehr Produktivität pro Nutzer, weniger Schulungsaufwand + geringere Fehlerquote (Layer 8)

- Herausforderung fürs Team: Verschiedene Nutzergruppen verstehen, frühe Einbindung der Nutzer, Usability-Tests
- Negative Auswirkungen: Geringe Nutzerakzeptanz, Produktivitätsverlust beim Kunden, hoher Schulungsaufwand

Softwarequalität

Softwarequalität gibt an, in welchem Maße ein Softwaresystem den Anforderungen an das System entspricht: Sowohl bzgl. des sichtbaren Verhalten der Software als auch in Bezug auf Organisation und Dokumentation

- Qualitätsmerkmale (FEBEWÜ)
  - ↳ Funktionalität → Korrektheit → Akzeptanztest
  - ↳ Zuverlässigkeit → Fehlertoleranz → Mean Time Between Failures
  - ↳ Benutzbarkeit → Bedienbarkeit → Usability-Test
  - ↳ Effizienz → Verbrauchsverhalten → Lasttest + Zeitmessungen
  - ↳ Wartbarkeit → Veränderbarkeit → Aufwand (Zeit) Bugfix
  - ↳ Übertragbarkeit → Anpassbarkeit → Portierung OS

CMMI

Capability Maturity Model Integration - Bewertet Reifegrad des Software-Entwicklungsprozesses eines Unternehmens auf Skala 1-5

- Wirkung: (i) Selbstauskunft: Ist-Stand → Weiteres Vorgehen
  (ii) Außendarstellung: Reifegrad als Wettbewerbsvorteil
- Kritik: (i) Hoher Aufwand & Kosten (nicht optimal für kleine Firmen)
  (ii) Keine direkte Aussage über Qualität konkreter Prozesse/Produkte

Vorgehensmodelle

Ein Vorgehensmodell strukturiert den Vorgang der Software-Erstellung
=> Rollen, Aktivitäten, Resultate/Dokumente

Wasserfallmodell

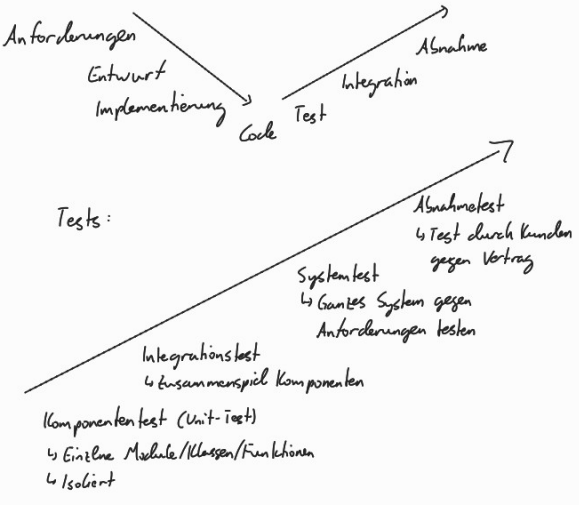
Klassisches Phasenmodell:

1) Anforderungsanalyse → Lastenheft
2) Systementwurf → Pflichtenheft, Architektur
3) Implementierung → Quellcode
4) Test → getestete Software
5) Betrieb / Wartung

=> Jede Phase abschließen, bevor nächste beginnt

V-Modell + Einsatz

Erweiterung Wasserfallmodell um korrespondierende Testumgebungen



- pro:
  - ↳ Erfasst auch zusätzliche Systemkomponenten
  - ↳ Standardisiert, umfangreich
  - ↳ Qualitätssicherung & Konfigurationsmanagement integriert
- contra:
  - ↳ Starke Reglementierung (Bürokratie)
  - ↳ Geringe Flexibilität gegenüber Anforderungsänderungen

Agiles Modell

Alternativer Entwicklungsansatz mit Fokus auf Anpassungsfähigkeit:

(i) Einzelpersonen & Interaktionen > Prozesse & Werkzeuge

(ii) Laufende Systeme > Umfangreiche Dokumentationen

(iii) Zusammenarbeit Kunde > Vertragsverhandlungen

(iv) Reagieren auf Änderungen > Verfolgung eines Plans

Scrum + Einsatz

Team selbstorganisiert im Vordergrund; Arbeitsetappen „Sprints"

▷ Scrum-Rollen:

(i) Product Owner (PO) - „Was?"
- ↳ Pflegt das Product Backlog (Liste aller Anforderungen)
- ↳ Priorisiert Anforderungen nach Geschäftswert
- ↳ Nimmt Sprint-Ergebnisse ab
- ↳ Schnittstelle zum Auftraggeber

(ii) Scrum-Master (SM) - „Wie?"
- ↳ Sorgt für korrekte Umsetzung Scrum-Prozess
- ↳ Beseitigt Hindernisse für das Team
- ↳ Schützt Team vor Störungen von außen

(iii) Entwicklungsteam - „Wer?"
- ↳ Setzt Sprint-Backlog-Items um
- ↳ Selbstorganisierend
- ↳ Pflegt Sprint-Backlog täglich

▷ pro:
- ↳ Hohe Flexibilität
- ↳ Kommunikation
- ↳ Kundennähe (Kunde eingebunden)

▷ contra:
- ↳ Schwierig bei sicherheitskritischen Systemen (wenig Doku)
- ↳ Setzt erfahrene Mitarbeiter voraus
- ↳ Kunde muss permanent verfügbar sein

Aufwandsabschätzung

Function Point Methode

FPM misst Software-Größe in Code-Komplexität und Funktionen
=> Funktioniert direkt nach der Anforderungsanalyse ohne Code

1) Anforderungen den Komponenten zuordnen

FP - Function Point - künstliche Maßeinheit für Funktionalität & Aufwand
UFP - Unadjusted FP - Summe aller FP ohne Einflussfaktoren
TFP - Total FP - Ergebnis inkl. Einflussfaktoren

EI - External Input; EO - External Output; EQ - External Query;
ILF - Internal Logical File; ELF - External Logical File

DE - Datenelement - Einzelnes Feld / Datenattribut
Records / References - Anzahl beteiligter Datentypen / Beziehungen

2) Komplexität bestimmen
- ↳ einfach / Mittel / Komplex => Aus Records + Datenelemente via Tabelle

3) Punktwert je Funktion via FP-Tabelle

4) UFP - Alle Werte addieren = S1

5) Einflussfaktoren (Gegeben) = S2

6) Korrekturfaktor: S3 = 0,7 + 0,01 · S2

7) Ergebnis: TFP = S1 · S3

COnstructive COst MOdel

CoCoMo schätzt Arbeitsaufwand in Personenmonaten aus Programmgröße

$A = a \cdot (KDSI)^b \cdot m(x)$

A - Aufwand in Personenmonaten (PM)

a - Konstante; Vorfaktor / Schwierigkeit via Tabelle

KDSI - Kilo Delivered Source Instructions = tausend ausgelieferte Quellzeilen

b - Exponent via Tabelle

m(x) - Multiplikator von Einflussfaktoren (z.B. Team-Erfahrung); Basisfall = 1

1) Programmgröße aus Lines of Code (LOC)
- ↳ Falls via FP: LOC = FP · LOC-pro-FP

2) KDSI = LOC / 1000

3) Schwierigkeit (a,b) aus Tabelle KDSI ↔ Schwierigkeit => a,b

4) m(x) = 1, Basisfall

5) Berechnung $A = a \cdot KDSI^b \cdot m(x)$

Optional:

TDEV - Time for DeveLopement; Kalenderzeit in Monaten: $TDEV = 2{,}5 \cdot A^c$

Nötige Personenanzahl: A / TDEV

Testverfahren

Statische vs. dynamische Tests

▷ Statische Testverfahren: Der Prüfling (zu prüfendes Dokument / Programm) wird nicht ausgeführt; Analyse erfolgt durch Lesen oder Werkzeuge, die Code analysieren ohne starten
- ↳ Inspektion / Walkthrough, Statische Codeanalyse, Formale Verifikation

▷ Dynamische Testverfahren: Prüfling wird mit konkreten Eingabewerten ausgeführt; Verhalten wird beobachtet
- ↳ Unit-Tests, Integrationstest, Systemtest, Akzeptanztest

=> Statische Tests können keine Laufzeitfehler (z.B. Endlosschleifen bei bestimmten Eingaben) erkennen oder falsche Parametertypen bei dynamisch typisierten Sprachen (z.B. Python), die erst zur Laufzeit bekannt sind

Äquivalenzklassen & Grenzwertanalyse

Form des Black-Box-Testen (Tests nur gegen Eingaben, nicht gegen den Code) bei der Eingabebereich in Klassen eingeteilt wird; Für Eingaben einer Klasse verhält sich der Code gleich → Pro Klasse ein Repräsentator-Testfall

=> Grenzwertanalyse untersucht Grenzen der Äquivalenzklassen:
Prüfe Wert vor, auf & nach Grenze

=> An Grenzen häufigste Fehler; Off-By-One, < statt ≤, Überläufe, ...

Fehlerfreiheit + Limits

Das Finden aller Fehler ist auch unter Auswahl optimaler Parameter nicht möglich, da aller Parameter in jeder Kombination ausprobiert werden müssten
→ ggf. unendliche Kombination
=> Nur gewisse Wahrscheinlichkeit, dass Fehler gefunden werden